

Why the interviewer does not sound like a person

A latency and turn-taking study of the Master Leeter voice path

Scope: `main` @ `e7f2dcb`

Written: 2026-09-13

Status: research and recommendation. No code changed.

Abstract

The voice interviewer on `main` is architecturally correct and conversationally slow. Its silence is a deliberate, well-argued product invariant; its *lag* is an accident of how that invariant was implemented. This paper separates the two.

Reading the code on `main` against the numbers in `ADR-001`, a MOCK-mode probe issued after a think-aloud turn cannot begin producing audio sooner than **~2.4 seconds** after the candidate stops speaking, and in practice lands between **3.9 and 5.2 seconds**, because four network round trips are serialized into the one moment where human listeners expect roughly **200 milliseconds**. Three of those four round trips are avoidable without weakening a single invariant in `ADR-001` or `CLAUDE.md`.

The second finding is that latency is not the whole complaint. Human conversation is not silence punctuated by speech; it is a continuously signalled channel — breath, room tone, continuers, prosodic turn-yielding. `main` has exactly two states, digital silence and a fully formed authored sentence, and the persona explicitly forbids everything in between. Even at zero latency, that reads as a machine waiting for input rather than a person sitting across the table.

The paper ends with a six-stage plan. The largest single win — and the one nobody seems to have noticed — is that **the interviewer's speech is entirely authored content**. Every word it is permitted to say comes from a reviewed scenario file through a tool that refuses anything else. Authored, finite text does not need a realtime generative model to speak it. It can be synthesized once, at session start, and played from a buffer in single-digit milliseconds. The realtime model is genuinely needed for *listening*, not for talking.

1. The complaint, stated precisely

The reported symptom is "too laggy, pauses too much, does not feel like a person." That is three distinct failures with three different fixes, and they must not be treated as one:

#	Symptom	Measurable quantity	Where it is caused
F1	It takes too long to answer	Response latency: last candidate sample → first audible interviewer sample	<code>policy.ts</code> thresholds, <code>turn-completion.ts</code> ramp, the tool relay, provider TTFA
F2	It pauses in the wrong places, or forever	Missed-response rate; false-cutoff rate	<code>vad.ts</code> , <code>gemini-classifier.ts</code> , <code>gate.ts</code> rules 2-3
F3	It does not feel like a person even when it is fast	Perceived naturalness; presence	<code>persona.ts</code> , absence of backchannels, absence of room tone, prosody

F1 is a measurement and engineering problem and is fully solvable today. F2 is a modelling problem with good off-the-shelf solutions. F3 is a design problem that the current persona actively works against, and it is the one that will still be there after the milliseconds are fixed.

A note on framing, because the codebase will resist this paper otherwise. `CLAUDE.md`, `gate.ts` and `turn-completion.ts` are emphatic that *silence is the product*, and they are right. Nothing below asks the interviewer to talk more. Every recommendation preserves `STAY_SILENT` as a first-class action and the Response Gate as the sole authority on whether speech happens. What they change is **how long it takes to execute a decision that has already been made**, which is a completely different variable and one the current design never separated out.

2. What "natural" means, quantitatively

Human turn-taking timing is one of the better-measured phenomena in conversation analysis, and the numbers are unforgiving.

- The modal gap between turns in conversation is around **200 ms**, and 51–55% of all transitions occur at under 200 ms across the corpora surveyed by Heldner and Edlund (2010), reported in Levinson and Torreira (2015).
- Measured over telephone conversations, Brady (1968) found mean gaps of **345–456 ms** with medians of **264–347 ms**.
- Across ten unrelated languages, Stivers et al. (2009) found mean response offsets to polar questions ranging from **7 ms to 468 ms**, with modes between **0 and 200 ms** — cross linguistically stable enough that the authors argue for a universal one-at-a-time norm with a strong preference for minimal gap.
- Overlap, when it happens, has a modal duration around **96 ms**, and 75% of overlaps are under 374 ms. Overlap is normal. It is not a failure state.

Now the part that matters for architecture. Producing even a single word takes roughly **600 ms** of planning before speech onset; a full sentence takes about **1500 ms** (Indefrey and Levelt 2004; Griffin and Bock 2000, both via Levinson and Torreira 2015). Breathing preparation adds several hundred more.

A 200 ms gap is therefore **arithmetically impossible** as a reaction. Humans do not react to the end of a turn; they **predict** it, and begin planning and articulating their response while the other person is still talking. Comprehension and production overlap.

This is the single most important finding in this paper, and `main` violates it structurally:

Every component in the current voice path begins its work **after** the candidate has finished speaking. The VAD waits for the hangover. The classifier waits for a finalized transcript. The gate waits for the classifier. The re-evaluation timer waits for the policy clock. The model waits for the authorization. The model then waits for its own tool call. Nothing is ever done in advance.

A system that only ever begins after the end of a turn is a system whose floor is the sum of its parts. A person's floor is the length of the *last* part, because the earlier parts already happened. That is the gap to close, and closing it is mostly a scheduling question rather than a model question.

Recent work on full-duplex dialogue models makes the same point from the other end: turn-taking decisions in Moshi-class models are predictable **more than 500 ms in advance** from the model's own internal state (arXiv 2605.20356). The signal exists before the silence does. Nothing in `main` reads it.

3. Where the time goes on `main`

3.1 The serialized chain

Tracing one authorized probe from the candidate's last syllable, with file references:

T0 — quiet onset. The candidate stops speaking. This is the moment the human clock starts.

T0 + 350 ms — VAD declares the end. `apps/web/lib/vad.ts`, `endHangoverMs: 350`. The event's `atMs` is correctly backdated to the quiet onset, so `silenceMs` is not inflated — but the *wall clock* has still advanced 350 ms before anything downstream knows a turn ended. This is detection lag, and it is unavoidable in principle, though its size is a choice.

T0 + ~400 ms — `activityEnd` reaches the provider, and `SPEECH_STOPPED` reaches the server over the app WebSocket.

T0 + ? — the transcript finalizes. This term is currently **unmeasured**, and it is likely to be one of the larger ones. The gate refuses to act on an unfinalized turn (rule 2 in `gate.ts`), so nothing can proceed until a final transcript exists. For reference, Speechmatics report that conventional streaming engines hold back finals for a fixed **700–1000 ms** of silence before emitting them, with processing on top; their "force end of utterance" path finalizes in roughly **250 ms** after an explicit client signal. Whether Gemini Live's input transcription behaves like the former or the latter when automatic activity detection is disabled is not recorded anywhere in the repository. It should be the first thing instrumented.

+ ~300–800 ms — the classifier. `gemini-classifier.ts` makes a structured JSON call to Gemini Flash-Lite once per finalized turn. The code comments this honestly: *"A model classifier suspends here for a few hundred milliseconds."* It is the only await between a turn arriving and the gate ruling.

T0 + 2425 ms — the silence hold expires. This is the big one, and it is worth showing the arithmetic because it is invisible in the source. `turn-completion.ts` caps end-probability at `HELD_FLOOR_CEILING = 0.35` until `minTurnEndSilenceMs`, then ramps linearly to 1.0 at `settledTurnEndSilenceMs`. To clear MOCK's `endOfTurnThreshold` of 0.80, the ramp must reach 0.80, which requires:

```
required = min + ((0.80 - 0.35) / (1 - 0.35)) × (settled - min)
          = 1500 + 0.6923 × (2800 - 1500)
          = 1500 + 900
          = 2400 ms
```

`holdForSilence()` then schedules re-evaluation at `required - silenceMs + 25`, so the effective floor is **2425 ms of quiet** before a non-question turn can be answered at all. The same computation for the other modes:

Mode	threshold	min	settled	Required silence
LEARNING	0.75	1200	2400	1939 ms
MOCK	0.80	1500	2800	2400 ms
STRICT	0.88	2200	4000	3668 ms

Note that the classifier's few hundred milliseconds are *absorbed* into this wait rather than added to it, as long as classification finishes before the hold expires. That is the one place where the current design already overlaps work. It is also the only one.

+ **~30–80 ms — authorization crosses to the client.** The server mints an `utteranceId`, `onAuthorized` fires, the client's `speak()` calls `requestSpeech()`.

+ **~600–1000 ms — the model's first response.** But here is the trap: the first thing the model produces is **not audio**. `instructionFor()` in `realtime-voice.ts` tells it to call `get_probe_wording` and say what comes back. So the first model output is a function call.

+ **~50–150 ms — the tool relay.** `relayToolCall()` sends the call to the *browser*, which forwards it to the server (`callTool`), which checks the authorization and returns the authored wording, which the browser sends back as a `toolResponse`. Three hops through a machine the candidate controls, to fetch a string the server already had in its hand at the moment it authorized.

+ **644–1958 ms — first audio byte.** ADR-001 measured this directly on `gemini-2.5-flash-native-audio-latest`, n = 20: **p50 1044 ms, p95 1638 ms**. That was for a simple `client_content` → audio turn with no tool hop. With the tool hop, the model takes two turns, so this cost is paid substantially twice.

+ **lead time — playback.** `playback.ts` schedules with a small deliberate lead to avoid clipping. Negligible, and correct.

3.2 The budget, assembled

Stage	MOCK probe (after think-aloud)	MOCK answer (direct question)
VAD hangover	350 ms	350 ms
Transcript finalization	unmeasured (250-1000 ms)	unmeasured (250-1000 ms)
Classifier	absorbed into hold	300-800 ms
Policy silence hold	2425 ms	0 ms (floor-yielded intent)
Authorization hop	30-80 ms	30-80 ms
Model turn 1 → tool call	400-900 ms	400-900 ms
Tool relay round trip	50-150 ms	50-150 ms
Model turn 2 → first audio	644-1958 ms	644-1958 ms
Total	≈ 3.9 - 5.2 s	≈ 1.7 - 3.3 s

Against a human baseline of ~200 ms modal and ~350 ms mean, the *best* case on `main` is five times too slow and the common case is roughly fifteen times too slow. That is not a tuning problem; it is a structural one, and it will not be fixed by lowering a threshold.

The `FLOOR_YIELDED_INTENTS` exemption in `turn-completion.ts` — questions and hint requests skip the silence gate entirely — is the single best decision in the current design, and it shows in the table above. The people who wrote it understood the problem. They applied the insight to one branch.

3.3 The tool round trip deserves its own paragraph

The authored-content-through-tools design exists for a good reason: the candidate must not be able to extract probe wording, hints, or the private solution from a browser they control. The server checks the authorization before it releases text.

But the current implementation achieves that property by making the **model** fetch the text, through the **browser**, at **speaking time**. The same security property is available by having the server attach the authorized wording at the moment it authorizes — the browser relays an opaque instruction it cannot read ahead of, or better, never sees the text at all because the audio is rendered server-side (§6.4). The check is "did the gate authorize this utterance", and the gate already answered that question before the message was sent. Asking it again, one model turn and three network hops later, buys nothing and costs roughly a second.

`gpt-realtime` now supports asynchronous function calling natively — the conversation continues while a call is outstanding — which softens this on the OpenAI path but does not remove it.

3.4 A quiet bug: the hangover is below the provider's documented floor

Google's Live API documentation states that when automatic activity detection is disabled, **manual end-of-speech thresholds should be at least 500 ms** of silence, because shorter values fragment the audio segments and degrade transcription quality. Their recommended `silenceDurationMs` for automatic VAD is **500-800 ms**.

`DEFAULT_VAD_CONFIG.endHangoverMs` is **350 ms**.

The reasoning in `vad.ts` for choosing 350 is sound in isolation — don't double-count patience with the policy layer — but it was reasoned against the *policy*, not against the *provider's transcription pipeline*. If segments are being cut short, the transcripts the classifier sees are worse than they should be, which lowers `textEndProbability`, which produces more held turns, which produces exactly the symptom under investigation. This is testable in an afternoon and it is the cheapest thing in this document to check.

3.5 The other half of the complaint: silence with no texture

`persona.ts` instructs: *"Say nothing. Silence is the normal state of this role and it is not awkward — the candidate is working. Do not fill it, do not acknowledge, do not check in."* The prohibition list forbids praise, evaluation, summarizing, and teaching.

As interview design, this is right. As **audio** design, it produces something no human interview has ever contained: a channel that is bit-exact digital silence for minutes at a time, and then a complete, perfectly fluent, two-sentence utterance from nowhere.

Real listeners emit continuers — "mm-hm", "right", "okay" — and these are not evaluative. In conversation analysis a continuer explicitly *declines* to take the floor; it means "keep going", which is precisely the message this interviewer wants to send and currently cannot. Deepgram report that in their production conversation data, listeners backchannel roughly **seven times more often than they interrupt**. A listening human makes noise.

Two consequences:

1. **The candidate cannot tell the system is alive.** There is no evidence of presence between utterances — no room tone, no breath, no acknowledgement. Long silence on a channel with zero noise floor reads as a dropped call, not as patience. This is likely a large part of "it doesn't feel like a person" independent of any millisecond.
2. **Every utterance arrives cold.** A person about to speak is audible before they are intelligible — an inbreath, a shift, a "so". `main` goes from absolute silence to mid-sentence fluency, which is precisely the quality people describe as robotic even when the content is perfect.

Note also that fillers are not merely cosmetic: natural speech is 5–10% disfluent, and filled pauses have been shown to *aid* listener recall relative to duration-matched non-speech sounds. Placement matters — medial fillers reduce perceived speaker confidence most — so the design should favour utterance-initial hesitation over mid-sentence stumbling.

4. Prior art: how comparable systems reach 200–800 ms

4.1 Model-based turn detection has replaced silence thresholds

The industry moved off "wait N milliseconds of silence" in 2025–2026, for exactly the reason `turn-completion.ts` articulates: a pause is not a turn end, and words alone cannot tell them apart. The difference is that everyone else concluded *use a better detector*, whereas `main` concluded *wait longer*.

- **Pipecat `smart-turn-v3`** — an 8M-parameter Whisper-Tiny-encoder classifier, int8 quantized to an **8 MB** artifact, **12 ms inference on a modern CPU** and 60 ms on a cheap AWS instance, across 23 languages. Weights, training data, and training script are open source. It consumes raw audio, not a transcript, so it does not wait for the STT pipeline at all.
- **LiveKit turn detector v1** — a dual-branch model: a semantic branch projecting audio into a language model's embedding space, and a separate encoder into a recurrent layer that captures **timing and prosody**, fused for the prediction. Reported **9.9% false-cutoff rate at 300 ms of latency** (vs 12.9% for the nearest competitor) and **4.5% at 600 ms** (vs 9.9% for Deepgram Flux), across 14 languages. Crucially, it reads audio directly and therefore "eliminates the latency cost of waiting for a transcript."
- **Deepgram Flux** — fuses transcription and end-of-turn detection into one model. `EndOfTurn` fires within 1.5 s in 95% of cases, with median latency **200–600 ms better** than a VAD+STT+endpointer pipeline, and roughly **30% fewer false interruptions**. Its `EagerEndOfTurn` event arrives **150–250 ms before** the confirmed one, explicitly designed for speculative LLM calls, at the cost of 50–70% more inference requests.
- **OpenAI `semantic_vad`** — server-side turn detection that chunks on *semantic* completion rather than silence, with an `eagerness` dial (`low` / `medium` / `high` / `auto`) trading interruption risk against wait time.

Every one of these does what `turn-completion.ts` wants to do, with prosody as an input rather than a fixed clock as a proxy for it. The prosodic information that distinguishes "I'll use a hash map" (finished) from "I'll use a hash map..." (continuing) is present *in the audio*, in pitch contour and final lengthening, which is exactly what a text classifier plus a stopwatch cannot see. `turn-completion.ts` even names this limitation and then works around it with patience. The fix is to feed the model the signal it is missing.

4.2 Nobody waits for the end any more

`EagerEndOfTurn`, `semantic_vad` eagerness, and speculative LLM invocation are all the same idea: start the expensive work on a *prediction* that the turn is ending, and discard it if wrong. This is the engineering analogue of the human prediction finding in §2, and it is the key to sub-second response.

The price is wasted inference — Deepgram quantify it at 50–70% more calls. For a product running one interview at a time, that price is irrelevant. For a product whose classifier is on a free-tier quota (as `gemini-classifier.ts` documents), it needs a budget check, but the authored-audio approach in §6.4 removes the expensive half of the speculation entirely.

4.3 Provider floor: time to first audio

Independent measurements of speech-to-speech models (Artificial Analysis, time-to-first-audio) show a wide spread that directly bounds what any architecture on top can achieve:

Model	TTFA
Deepslate Opal	0.44 s
Gemini 2.5 Flash Native Audio Dialog	0.63 s
Grok Voice Think Fast 2.0 High	0.70 s
GPT-Realtime-1.5	0.81 s
GPT-Realtime-2 (High)	1.14 s
Amazon Nova 2.0 Sonic	1.14 s
Gemini 3.1 Flash Live High	2.99 s

Two observations. First, `ADR-001`'s measured p50 of 1044 ms on Gemini 2.5 Flash native audio is noticeably worse than the 0.63 s benchmark for the same family, which suggests the spike's figure includes round-trip and setup overhead worth re-measuring from the browser rather than from a Node script on a home connection. Second, **the newest is not the fastest**: a naive upgrade to Gemini 3.1 Flash Live would nearly triple the provider floor. Model selection for this product should be driven by TTFA, not by capability headlines.

4.4 Filler, backchannel, and presence as latency *masking*

When latency cannot be removed, it can be covered. LiveKit's own latency guidance recommends playing a "thinking" sound during tool execution so the user is not left without feedback. Voice-agent vendors ship filler-word insertion for the same reason. A CHI 2025 study on LLM-powered voice agents for older adults found that explicit support for **interruptions and backchannels** was central to perceived naturalness — not response speed alone.

For an interview specifically, the available non-evaluative signals are: a continuer ("mm-hm"), an inbreath before speaking, and a constant low room tone. None of them are assessment. All three are things the current persona forbids or omits.

4.5 The research frontier: full duplex

Moshi and successors model listening and speaking as simultaneous streams rather than alternating turns, which dissolves the endpointing problem rather than solving it. The relevant near-term finding is the one cited in §2: turn boundaries are predictable hundreds of milliseconds in advance from audio alone. This is not a suggestion to adopt a full-duplex model — the gate architecture depends on discrete authorized utterances — but it is strong evidence that a small predictive model can supply what the current fixed clock is standing in for.

4.6 Comparable products

The AI-interviewer category (Apriora, micro1, Mercor's interview agent, and the rest of the 2026 cohort) is largely built on the same commodity stack: a realtime speech-to-speech model or a VAD+STT+LLM+TTS cascade, wrapped in an interview script. None of them, as far as public material shows, publish turn-taking latency figures. What is worth noting is the shape of the competition: they are optimizing a *screening* interaction, where the agent talks most of the time and the candidate answers. Master Leeter is optimizing the inverse — the candidate talks almost continuously and the interviewer speaks rarely. That inversion is why the silence-first architecture is right, and also

why latency is *more* visible here, not less: when the interviewer only speaks eight times in forty minutes, every one of those eight entrances is scrutinized.

5. Diagnosis

Three design decisions, each locally defensible, compound into the reported symptom.

D1. Patience and lag were conflated. The policy layer expresses "this interviewer is unobtrusive" as "this interviewer waits 2.4 seconds before it is allowed to conclude you stopped talking." Those are different properties. A real interviewer who has decided not to interrupt you is silent for *minutes*; the same interviewer, having decided to speak, begins in 200 ms. The current model can only express reluctance as slowness. Reluctance belongs in the gate's decision; it must not appear in the execution path once the decision is `SPEAK`.

D2. Everything begins after the end. No component starts work on a prediction. The chain in §3.1 is fully serial, and because each stage waits for the previous one to produce a final artifact, the total is the sum. Humans achieve a 200 ms gap by making the sum irrelevant.

D3. The model fetches its own words at speaking time. A correct security property is implemented at the worst possible point in the timeline, costing an extra model turn plus three hops. The authorized text is known to the server before the client is even told to speak.

And one omission:

D4. There is no evidence of presence. Between utterances the channel carries nothing at all. The system has no way to say "still here, still listening, keep going."

6. Proposed architecture

Ordered by ratio of effect to risk. Each stage is independently shippable and each is checked against `ADR-001` and the `CLAUDE.md` invariants.

6.1 Stage 0 — Instrument before changing anything

Nothing below should be tuned against a guess. The event log already exists; add a per-utterance latency ledger with these timestamps, all client-side where the candidate's ear is:

```

t_quiet_onset      (VAD backdated atMs)
t_vad_end_detected (+hangover)
t_activity_end_sent
t_transcript_final ← currently unmeasured, suspected large
t_classified
t_decided          (gate returned)
t_authorized_rx    (client received authorization)
t_speech_requested
t_tool_call_rx / t_tool_response_sent
t_first_audio_byte
t_first_sample_played ← this is the number the candidate experiences

```

Define the product metric as `t_first_sample_played - t_quiet_onset` and report p50/p95 per action type. Add it to the existing eval harness. **Acceptance: one recorded session produces a full ledger for every utterance, and the three largest terms are named.**

This stage is non-negotiable. §3.2 contains one term marked "unmeasured" and several ranges; the plan below could be wrong about their relative size, and the ledger is what settles it.

6.2 Stage 1 — Replace the silence clock with a turn-end predictor

Keep `turn-completion.ts`'s interface, its `min`-only monotonicity property, and the mid-thought veto. Replace the *input*: instead of deriving `endProbability` from elapsed silence, derive it from a model that reads the audio.

Recommended: run **smart-turn-v3** (8 MB, ONNX, 12 ms CPU) client-side, beside the existing `Vad`, in the same worklet that already has the samples. This preserves ADR-001's requirement that the client owns turn-boundary detection on the Gemini path, adds no network hop, and adds no provider dependency. `silenceCeiling()` becomes `prosodicCeiling(audioWindow)`, and `minTurnEndSilenceMs` drops from 1500 to roughly 300–500 ms because the clock is no longer carrying the whole burden of distinguishing a breath from a yield.

Keep the ramp. It is good design for the same reason the file says it is — a discontinuity at one millisecond reads as inattention. Just make it a ramp over a *better-informed* probability.

Invariant check: unchanged. The gate still decides; this only improves the quality of one of its inputs. `STAY_SILENT` remains the default.

Acceptance: false-cutoff rate at fixed latency, measured on recorded sessions, no worse than today's at 2400 ms while the median hold drops below 600 ms. Report both numbers together — either alone is gameable.

6.3 Stage 2 — Decide during the turn, not after it

Run classification and the gate **speculatively on partial transcripts** while the candidate is still speaking, at a fixed cadence or on each interim transcript. Cache the resulting decision keyed by transcript prefix. When the turn-end predictor fires, the decision is usually already in hand, and the only work left on the critical path is the freshness check that `runtime.ts` already performs (`staleGroundingReason`).

This is the direct analogue of the human prediction finding, and of Deepgram's `EagerEndOfTurn`. It removes 300–800 ms of classifier latency from the path in the common case.

Costs and mitigations: more classifier calls on a metered quota. Mitigate by speculating only after the predictor's probability crosses a low water mark (say 0.4), by deduplicating on transcript prefix, and by keeping the existing circuit breaker. The rule-based fallback already handles quota exhaustion.

Invariant check: a speculative decision is *not* an authorization. Nothing is spoken until the real turn-end fires and the gate's final check passes on the finalized transcript. The log must record both the speculative and the confirmed decision so replay stays exact — this is the one place where Stage 2 could damage the replay property in `replay.test.ts`, and it needs a test of its own.

6.4 Stage 3 — Pre-render the authored audio (the big one)

Observation: by design, this interviewer never improvises. `persona.ts`: *"Everything substantive comes from a tool."* The tools return reviewed scenario content — the opening brief and its repeat variants, clarification facts, probe wordings, hint ladders, follow-ups. The set of things the interviewer can say in a given session is **finite and known at session start**, because the scenario version is pinned at creation (`002_session_storage.sql` makes the pin immutable).

Therefore: synthesize them all at session start, in the interviewer's voice, and cache the PCM server-side. When the gate authorizes, stream the cached audio. Time to first sample becomes **network latency plus buffer lead** — call it 50–150 ms — instead of 1.0–2.0 s of model generation plus a tool round trip.

This deletes, at a stroke:

- the model turn that produces a tool call (400–900 ms),
- the tool relay round trip (50–150 ms),
- the model turn that produces audio (644–1958 ms),
- and the entire class of bug where the model "lightly adapts" authored wording in a way nobody reviewed.

It also *strengthens* every invariant in `ADR-001` and `CLAUDE.md`. A model that cannot generate output audio cannot speak out of turn, cannot leak the problem statement, cannot praise, cannot teach, and cannot be prompt-injected into saying anything at all, because it is no longer in the output path. The gate's authorization becomes literally the thing that plays a buffer. "No config exists in which the model speaks without a gate decision" stops being a property maintained by careful credential configuration and becomes a property of the data flow.

What is lost, and must be weighed honestly:

- **Prosodic responsiveness.** A native-audio model can match the candidate's register. A pre-rendered line cannot. For an interviewer persona that is explicitly *"neutral and unhurried, not warm, not cold"*, this is a small loss and arguably an improvement in consistency.
- **Adaptive phrasing.** Already forbidden by the persona.
- **Anything genuinely generative.** There is currently nothing. If a future iteration wants an AI-restated question (I03 in the production plan), that restatement is generated *before* the interview and pinned, so it is still pre-renderable.
- **Synthesis cost and startup time.** A scenario's full utterance set is on the order of tens of lines. Cache by `(scenarioVersionId, voiceId, ttsModel)` so it is generated once per scenario version, not once per session.

Then it is free.

The realtime model is still needed — for input transcription, and for the small number of utterances that genuinely depend on what the candidate just said (the acknowledgement path, and any future clarification that must be phrased around the question). Run it as a listener with output suppressed, and treat the output channel as an escape hatch rather than the default.

A hybrid is the pragmatic landing point: **cached audio for the 90% of utterances that are verbatim authored content; live model for the remainder.** Measure the split from the existing decision logs before committing.

Acceptance: for any authored action, `t_first_sample_played - t_decided` under 200 ms p95.

6.5 Stage 4 — Give silence a texture

Three additions, none of which say anything evaluative:

1. **Room tone.** A continuous, very low-level ambient bed on the interviewer's channel for the whole session, present from connect. This is the cheapest change in this document and plausibly one of the most effective: it converts "is this thing on?" into "someone is sitting there." Absolute digital silence is not a sound that exists in a room with a person in it.
2. **Continuers.** A small set of pre-rendered non-evaluative backchannels ("mm-hm", "okay", "sure"), authorized by a separate cheap rule — not the main gate — during long candidate turns, at a low rate, never during the candidate's own pauses in a way that could be read as a floor bid. This requires a persona amendment: the current text forbids acknowledgement outright, and that prohibition was written to prevent *assessment*, which a continuer is not. The distinction is worth writing into `persona.ts` explicitly rather than leaving to interpretation.
3. **Onset cues.** A short inbreath or hesitation particle rendered ahead of authored lines, so utterances do not begin from nothing. Place them utterance-initially, never mid-sentence (medial fillers are the ones that read as low confidence).

These three are also, conveniently, latency *masking*: an onset cue that starts 150 ms before the substantive audio buys 150 ms of cover, and does so in a way that is indistinguishable from what a person does while drawing breath to speak.

Risk: overdone, this becomes the chatty assistant the persona exists to prevent. Ship continuers behind a rate limit and a config flag, and evaluate with human review before enabling by default. This is the one recommendation in the paper that could make the product worse if tuned badly.

6.6 Stage 5 — Backchannel-aware barge-in

`gate.ts` rule 1 and the client's `onBargeIn` both treat any candidate speech during interviewer audio as a floor bid. If the candidate says "mm-hm" while the interviewer is delivering the brief, the brief is cut off mid-word. Given the ~7:1 base rate of backchannels to interruptions, this fires wrongly far more often than it fires rightly.

Fix: classify the interrupting vocalization before cancelling. Short, affirmative, non-continued → hold and keep speaking. Negative tokens ("no", "wait", "uh-uh") or continued speech past ~600 ms → yield immediately, as today.

The immediate-cancel path must remain the default when classification is uncertain, because talking over someone is the worse error.

6.7 Stage 6 — Provider selection, measured

If the personal GPT-Live direction proceeds, note that OpenAI's `semantic_vad` with `create_response: false` provides exactly the ADR-001 invariant — server-side semantic turn detection that emits boundaries **without** answering — which is the capability ADR-001 recorded as the reason client-side VAD was necessary on Gemini. That combination would let Stage 1 be configuration rather than a model deployment. Weigh it against the TTFA table in §4.3 and re-measure from a browser, not from a script.

7. Target budget

Stage	Today (MOCK probe)	After Stages 1-4
Turn-end detection	350 ms VAD + 2425 ms policy hold	300-500 ms predictor
Transcript finalization	unmeasured, on the critical path	off the critical path (speculative)
Classifier	absorbed / 300-800 ms	~0 (pre-computed)
Decision → first audio	1100-3000 ms (2 model turns + relay)	50-150 ms (cached audio)
Total	3.9 - 5.2 s	≈ 400 - 700 ms

400–700 ms puts the interviewer inside the human gap distribution — slower than the 200 ms mode, comfortably inside Brady's 345–456 ms mean gap range, and well inside the range people describe as conversational. Combined with room tone and onset cues, that is the difference between "laggy" and "considered."

Note that this target is achieved **without reducing the interviewer's patience by a single decision**. The gate can still choose to stay silent exactly as often as it does today. What changes is that when it chooses to speak, it speaks.

8. How to evaluate

Latency and interruption quality trade off directly, so they must always be reported as a pair. A system can reach 100 ms by interrupting constantly, and 0% false cutoffs by never speaking. `main` currently sits at the second extreme.

Primary metrics, all logged per utterance and reported p50/p95:

- Response latency** — `t_first_sample_played - t_quiet_onset`, split by action type (`ANSWER_CLARIFICATION` and `DELIVER_BRIEF` are the ones a candidate actively waits on).
- False-cutoff rate** — interviewer speech that began while the candidate had not finished, as judged by human review of recorded sessions. Report *at a stated latency*, the way LiveKit and Deepgram do, or the number is meaningless.

3. **Missed-response rate** — direct questions that received no answer. Already a tracked metric in `CLAUDE.md`; it is the metric this whole paper risks damaging, so it gates everything.
4. **Backchannel false-stop rate** — interviewer utterances cancelled by a candidate vocalization that was not a floor bid.
5. **Perceived naturalness** — a short human rating on recorded sessions (presence, pacing, "did this feel like a person"). Three to five raters is enough to detect the effects this paper predicts; a single self-rating is not.

The existing eval harness (`apps/api/src/eval/`) is the right home for 1–4. Metric 5 requires the real session that `docs/ISSUES.md` has been honest about not yet having. **No number in this paper substitutes for that session**, and the first thing that session should produce is the ledger from §6.1.

9. What this paper does not know

Stated plainly, in the style the rest of the repository uses:

- **Transcript finalization latency is unmeasured.** It sits on the critical path and could be anywhere from 250 ms to over a second. If it is at the high end, Stage 0 changes the priority order of everything after it.
 - **The 3.9–5.2 s total is derived, not observed.** It is assembled from the code on `main`, the `ADR-001` spike numbers, and documented provider behaviour. No one has yet recorded a real session with a real microphone — `docs/ISSUES.md` says so repeatedly and it is still true. The derivation could be wrong in either direction.
 - **The tool round trip's cost is estimated.** A model that emits a function call and then audio may pipeline better than the two-full-turns assumption here.
 - **Whether the 350 ms hangover actually degrades Gemini's transcription is untested.** Google documents a 500 ms floor for manual VAD; whether the effect is material at 350 ms is an empirical question.
 - **Smart-turn-v3's accuracy on this specific population is unknown.** Candidates thinking aloud mid-problem produce long, hesitant, technically dense speech with many mid-thought pauses — which is close to the hardest case for any endpointer, and is not obviously well represented in general conversational training data. It may need fine-tuning on recorded sessions, which is another reason to start recording them.
 - **Pre-rendered audio has not been prototyped.** The claim that it removes 1.5–2.5 s rests on the assumption that streaming cached PCM through the existing playback scheduler is straightforward. It looks straightforward. It has not been done.
-

10. Implementation order

Each stage is a checkpoint in the style of `NEXT_IMPLEMENTATION_PLAN.txt` : one bounded change, tested, committed, owner reviews and merges.

#	Change	Files	Expected gain	Risk
V0	Latency ledger + eval metrics	<code>runtime.ts</code> , <code>voice-session.ts</code> , <code>eval/metrics.ts</code>	0 ms, enables everything	none
V1	Raise <code>endHangoverMs</code> to 500 and measure transcript quality	<code>vad.ts</code>	possibly better transcripts	+150 ms detection, offset by V2
V2	Turn-end predictor replaces the silence ramp	new <code>turn-predictor.ts</code> , <code>turn-completion.ts</code> , <code>policy.ts</code>	−1.8 to −2.0 s	false cutoffs; gated on metric 2
V3	Pre-rendered authored audio	<code>realtime/tools.ts</code> , <code>session/channel.ts</code> , <code>playback.ts</code> , new TTS cache	−1.5 to −2.5 s	voice consistency; strengthens invariants
V4	Speculative classification on partials	<code>runtime.ts</code> , <code>gemini-classifier.ts</code>	−0.3 to −0.8 s	quota; replay exactness
V5	Room tone + onset cues	<code>playback.ts</code> , <code>persona.ts</code>	perceived, not measured	low
V6	Continuers	<code>gate.ts</code> (separate cheap rule), <code>persona.ts</code>	perceived	can make it worse; flag it
V7	Backchannel-aware barge-in	<code>realtime-voice.ts</code> , <code>gate.ts</code>	fewer wrong cancellations	must fail toward yielding

V0 first, always. V2 and V3 are independent and together account for roughly 90% of the improvement; if only one can be done, **V3 is the cheaper and the safer**, because it removes latency without touching any decision boundary.

References

Research and documentation consulted for this paper.

Human turn-taking

- Levinson, S. C., & Torreira, F. (2015). *Timing in turn-taking and its implications for processing models of language*. *Frontiers in Psychology* 6:731. <https://www.frontiersin.org/journals/psychology/articles/10.3389/fpsyg.2015.00731/full> (Source for: 200 ms modal transition, 51–55% under 200 ms via Heldner & Edlund 2010; Brady 1968 gap means/medians; production latencies of ~600 ms/word and ~1500 ms/sentence via Indefrey & Levelt 2004 and Griffin & Bock 2000; the prediction argument.)
- Stivers, T., et al. (2009). *Universals and cultural variation in turn-taking in conversation*. *PNAS* 106(26). <https://www.pnas.org/doi/10.1073/pnas.0903616106> (Cross-linguistic response offsets; figures quoted here are as reported in Levinson & Torreira, since the PNAS page could not be fetched directly.)
- Synchronization and Turn-Taking in Full-Duplex Speech Dialogue Models*. arXiv 2605.20356. <https://arxiv.org/html/2605.20356>
- Kyutai Labs, *Moshi: a speech-text foundation model for real-time dialogue*. <https://kyutai.org/Moshi.pdf>

Turn detection and endpointing

- LiveKit, *Solving end-of-turn detection: Turn Detector v1*. <https://livekit.com/blog/solving-end-of-turn-detection>
- LiveKit, *Turn Detection for Voice Agents: VAD, Endpointing, and Model-Based Detection*. <https://livekit.com/blog/turn-detection-voice-agents-vad-endpointing-model-based-detection>
- Daily, *Announcing Smart Turn v3, with CPU inference in just 12 ms*. <https://www.daily.co/blog/announcing-smart-turn-v3-with-cpu-inference-in-just-12ms/>
- `pipecat-ai/smart-turn-v3` model card. <https://huggingface.co/pipecat-ai/smart-turn-v3>
- Deepgram, *Introducing Flux: Conversational Speech Recognition*. <https://deepgram.com/learn/introducing-flux-conversational-speech-recognition>
- Speechmatics, *You can't hurry love, but you can hurry final transcripts*. <https://www.speechmatics.com/company/articles-and-news/you-cant-hurry-love-but-you-can-hurry-final-transcripts>

Provider documentation and benchmarks

- OpenAI, *Realtime API — Voice activity detection (VAD)*. <https://developers.openai.com/api/docs/guides/realtime-vad>
- OpenAI, *Introducing gpt-realtime and Realtime API updates for production voice agents*. <https://openai.com/index/introducing-gpt-realtime/>
- Google, *Gemini Live API capabilities guide* (VAD configuration, manual activity detection, interruption handling). <https://ai.google.dev/gemini-api/docs/live-guide>
- Google, *Gemini Live API overview*. <https://ai.google.dev/gemini-api/docs/live-api>
- Artificial Analysis, *Speech to Speech Models and Providers Analysis* (time to first audio). <https://artificialanalysis.ai/speech-to-speech>
- LiveKit, *Understand and improve agent latency*. <https://livekit.com/blog/understand-and-improve-agent-latency>

Naturalness, backchannels, fillers

- Deepgram, *Backchannels vs Interruptions in Voice Agents*. <https://deepgram.com/learn/backchannels-vs-interruptions-voice-agents>
- *Toward Enabling Natural Conversation with Older Adults via the Design of LLM-Powered Voice Agents that Support Interruptions and Backchannels*. CHI 2025. <https://dl.acm.org/doi/full/10.1145/3706598.3714228>
- Rime, *How to Add Natural Filler Words to TTS Voice Agents*. <https://www.rime.ai/resources/how-to-add-natural-filler-words-to-tts>

Internal (this repository, main @ e7f2dcb)

- `docs/adr/ADR-001-response-control.md` — provider spike, latency samples, invariant.
- `apps/web/lib/vad.ts`, `apps/web/lib/audio.ts`, `apps/web/lib/playback.ts`, `apps/web/lib/realtime-voice.ts`, `apps/web/lib/voice-session.ts`
- `apps/api/src/modules/orchestrator/{policy,gate,turn-completion,runtime,gemini-classifier}.ts`

- `apps/api/src/modules/realtime/{persona,token,tools}.ts`
- `CLAUDE.md`, `docs/ISSUES.md`, `TODO.txt`